

Day 9 — CPI Palette Functions (25 Aug 2026)

Spelling correction throughout: Your notes use "Pallet(s)" — the correct term is **Palette** (as in, a toolbox/panel of available steps). "Pallet" is a shipping/logistics term and unrelated. Corrected everywhere below.

1. Recap — What Is CPI?

CPI → Middleware Service. It transfers data from one system to another system.

2. What Is Data? — Two Steps for Message Processing

Step	Description
1. Connectivity	Establishing the connection itself — e.g., HTTPS, Gmail (Mail)
2. Message Transformation	Converting/modifying the data as it passes through

3. Palette Options — Where They Live

Palette options → Message Transformation → Integration Process. The Palette is the panel of available steps you drag onto your Integration Process canvas.

4. Why Palette (Participants) — What We Have

① Participant



A Participant represents either:

- **Sender (Source)**
- **Receiver (Target)**

It represents what is the **Source System and Target System.**

5. ② Process — Three Types

Process	→	Exception Subprocess
	→	Integration Process
	→	Local Integration Process

iFlow Structure

iFlow → Main Body → Subprocess — all of these are included in one Package.

Integration Process = "Main Process" — this is the primary, top-level process every iFlow has.

Local Integration Process — Why It Exists

Local Integration Process → If we want to use the same logic multiple times, then we will use Local Integration Process.

It's called using **Process Call** — a Palette function.

Diagram — Main Flow Calling a Local Integration Process



(Process Call step branches down to invoke:)

Local Integration Process (its own start/end events)

6. Exception Subprocess

Event Shape Differences — Integration Process vs. Local Integration Process

Process Type	Start Shape	End Shape
Integration Process	Start Message (square)	End Message (square)
Local Integration Process	Start Event (circle)	End Event (circle)

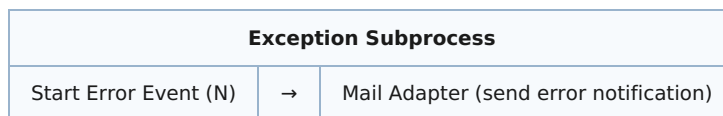
Handy visual rule to remember: Square shapes = Message events (used in the main Integration Process). Circle shapes = plain Events (used in Local Integration Process and Exception Subprocess). This shape difference is a quick way to recognize, just by looking at a canvas, which type of process you're looking at.

What Exception Subprocess Does

Catches errors and handles exceptions during execution. If any failure occurs, the Exception Subprocess will be invoked automatically.

Important rule: Exception Subprocess will be mandatorily used in Main Integration Process (or) Local Integration Process — other than these two, we cannot use it. (i.e., Exception Subprocess isn't a standalone, freely reusable component like Local Integration Process — it only exists attached to one of these two process types.)

Worked Example — Exception Subprocess with Mail Adapter



(This ties directly back to Day 8's Mail adapter setup — a very common real pattern: when something fails, automatically email someone about it.)

7. ③ Events

Full list of event types covered:

- End Event
- End Message
- Error End Event
- Error Start Event
- Escalation End Event
- Start Event
- Start Message
- Terminate Message
- Timer

Core purpose: Events decide when the process should Start and End.

Timer — Important Detail

Timer → iFlow triggers → Schedule Time. This connects to a **Scheduler** — flagged in your notes as **Important**. Timer-based start events let an iFlow run automatically on a defined schedule (e.g., every night at 2 AM) rather than being triggered by an incoming message.

End Message

End Message → End of Message Transformation process.

Diagrams — Event Pairs by Process Type

--	--	--

Process	Start	End
Integration Flow	Start Message (envelope icon)	End Message (envelope icon)
Local Integration Process	Start Event (circle)	End Event (circle)
Exception Subprocess	Start Error Sub-process (N)	End Error/End Event

8. ④ Call

→	Connectivity Part
---	-------------------

9. ⑤ Delete (Lock Icon)

Shown in your notes as a simple delete function within the palette.

10. ⑥ Mappings

Mappings → Data format or structure will be modified using mappings.

Mapping Type
ID Mapping
Message Mapping
Operation Mapping
XSLT Mapping

Real-world usage: In real-time projects, we will mostly use **Message Mapping** and **XSLT Mapping** — the other two are less commonly used day-to-day.

11. ⑦ Transformations — Full List

Function	Notes
Content Modifier	Used to modify the incoming payload — supports Add, Delete, Convert, Secure operations
Converter	EDI ↔ XML, CSV ↔ XML, JSON ↔ XML
Decoder	Includes ZIP handling
EDI Extractor	
Encoder	
Filter	
Message Digest	
Script	Java or Groovy
XML Modifier	

12. Converting Non-XML Data to XML — Important Concept

JSON → XML → Now we will use Palette (options).

Important clarification — your notes say "All Palette [functions/options] only support XML." This needs nuance, and I researched this specifically to give you an accurate answer:

This statement is **mostly true for a specific, important subset of Palette functions** — but not universally true for every single Palette function in CPI. Specifically:

- Confirmed XML-only:** The **Aggregator** function only supports XML. Several routing/mapping functions (like certain Splitter modes, Message Mapping in its classic form) are also built around XML as their native working format.

- **Not XML-only:** Functions like **Router** and **Local Call → Looping Process Call** explicitly support a **"Non-XML" Expression Type** as a configurable option — meaning they can evaluate conditions against JSON or plain text directly, not just XML.
- **Content Modifier** works with any payload type (XML, JSON, plain text) — it doesn't require XML.

What your trainer likely meant, and the practical takeaway: A large, important core group of Palette functions — especially anything involving graphical **Message Mapping** — does expect/require XML as the working format internally. This is exactly *why* the JSON → XML → (process) → XML → JSON pattern you're about to see below is such a standard, common pattern in real CPI projects: you convert non-XML data to XML before mapping, then convert back afterward if the receiver needs JSON. So while "all Palette functions" is too broad a claim, the underlying practical habit your trainer is teaching you — convert to XML before serious processing/mapping — is correct and important to internalize.

13. ⑧ Call — External vs. Local

Call	→	External Call
	→	Local Call

External Call — Sub-options

- Content Enricher
- Poll Enrich
- Request Reply
- Send (*only Request — No Response*)

When to Use External Call

In between the flow, if we want to get any data, we will use Request Reply. This is the standard pattern when your iFlow needs to reach out mid-process to fetch additional data from another system before continuing.

14. ⑨ Routing Conditions

Function	Notes
Aggregator	
Gather	
Join	
Multicast	Supports Parallel and Sequential processing modes
Router	
Splitter	Types: General, Iterating, IDoc, PKCS7/CMS; supports EDI, ZIP, TAR handling

15. ⑩ Security — Message Level Security

Core idea: Suppose data needs to go from readable → non-readable (or vice versa) for protection purposes.

Function
Decryptor
Encryptor
Signer
Verifier

16. ⑪ Persistence

Temporarily stores the data. Sub-functions:

- **Data Store Operation**

- Persist
- Write Variables

17. 12 Validator

- EDI Validator
- XML Validator

18. Now: Message Transformation Using Palette Options — Worked Example

Message → Transforms Data. Data can be pushed in many different ways: **CSV, Text, XML, JSON, encrypted data, encoded, ZIP, compressed, and even videos → text or audio → text** (via appropriate conversion/extraction steps).

Most commonly used formats: Text, CSV, XML, JSON.

So now we will convert non-XML data to XML by using the Converter option from Palette options.

Cardinality Note

Header (1) : N — meaning one header can relate to multiple (N) line items, a standard 1-to-many relationship pattern you'll see reflected in the XML structure below.

Example — Student Record

Plain Text / CSV format (Header + Line Items):

Field	Value
Roll No	1
Name	Ashok
Phone No	9923981 0
Address	Penna

Same Data Converted to XML (2 Students Example)

```
<Students>
  <Student>
    <RollNo>...</RollNo>
    <Name>...</Name>
    <PhoneNo>...</PhoneNo>
    <Address>...</Address>
  </Student>
</Students>
```

Same Data in JSON Format

```
[
  {
    "Name": "Bharath",
    "RollNO": "...",
    "phono": "...",
    "Address": "..."
  }
]
```

19. Demo: Content Modifier (Flagged as "Very Very Important")

Message → actual transfer data.

Every message consists of four parts:

1. **Message Header**
2. **Message Property**

3. Message Body

4. Attachments

Header — Practical Example

Header → CPI Message → Username/Password →

- **Authorization** → Encoded format
- **Content-Type** → Plain text

Properties

Properties → Variable → Storing a value in a container and reusing it again. (This matches exactly what we covered earlier — properties are internal-only, used to carry a value from one step to a later step within the same execution.)

20. Trace — Checking the Message Process (Recap)

Trace → if we want to check message process, [use Trace,] start to end.

Sender	→	Integration Process (Content Modifier → Envelope → Envelope → Content Modifier)	→	Receiver
--------	---	---	---	----------

(This reinforces Day 8's trace demonstration — being able to trace a message end-to-end through every step is one of the most important day-to-day debugging skills in CPI.)